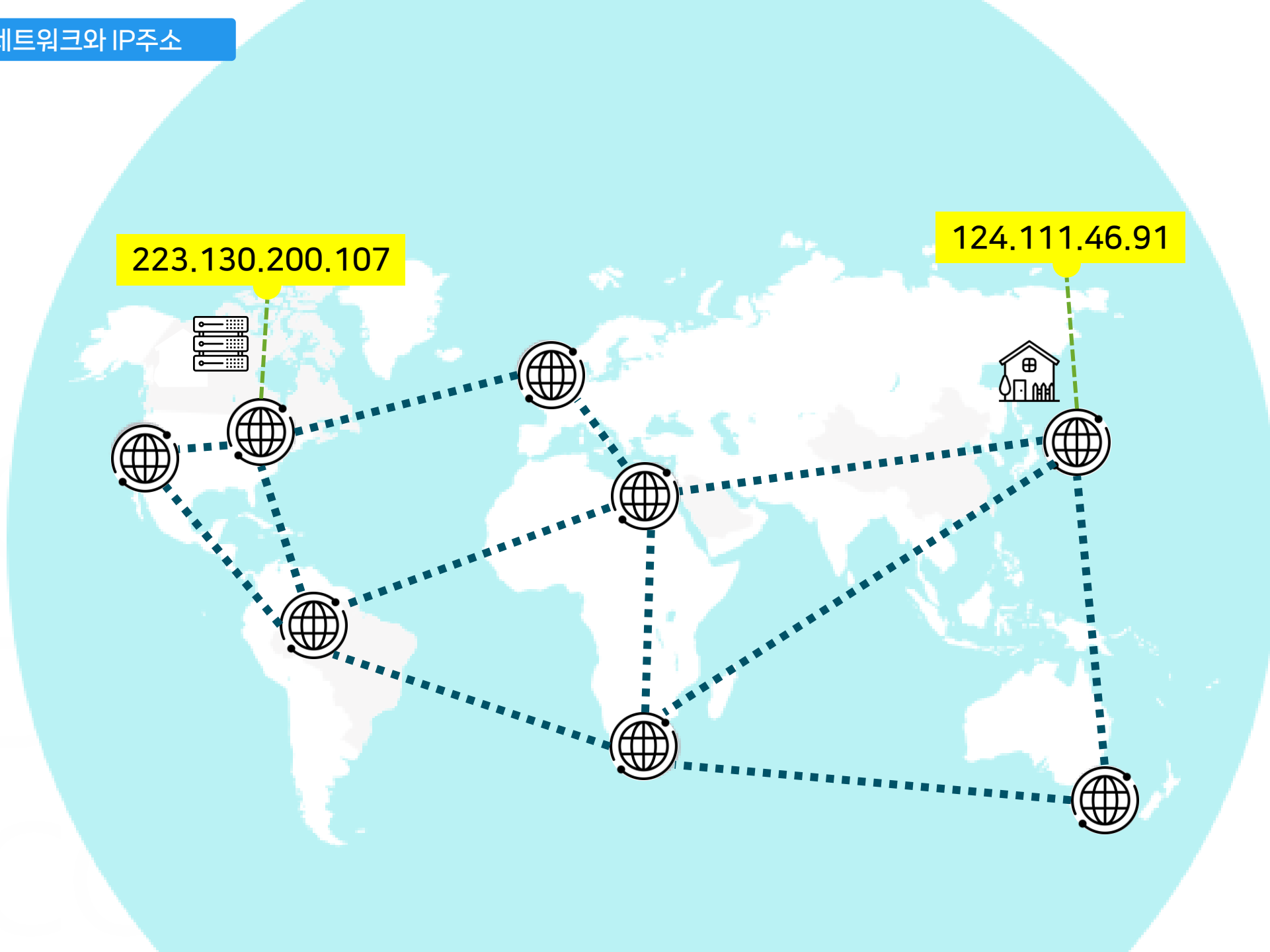


쉬운 도커

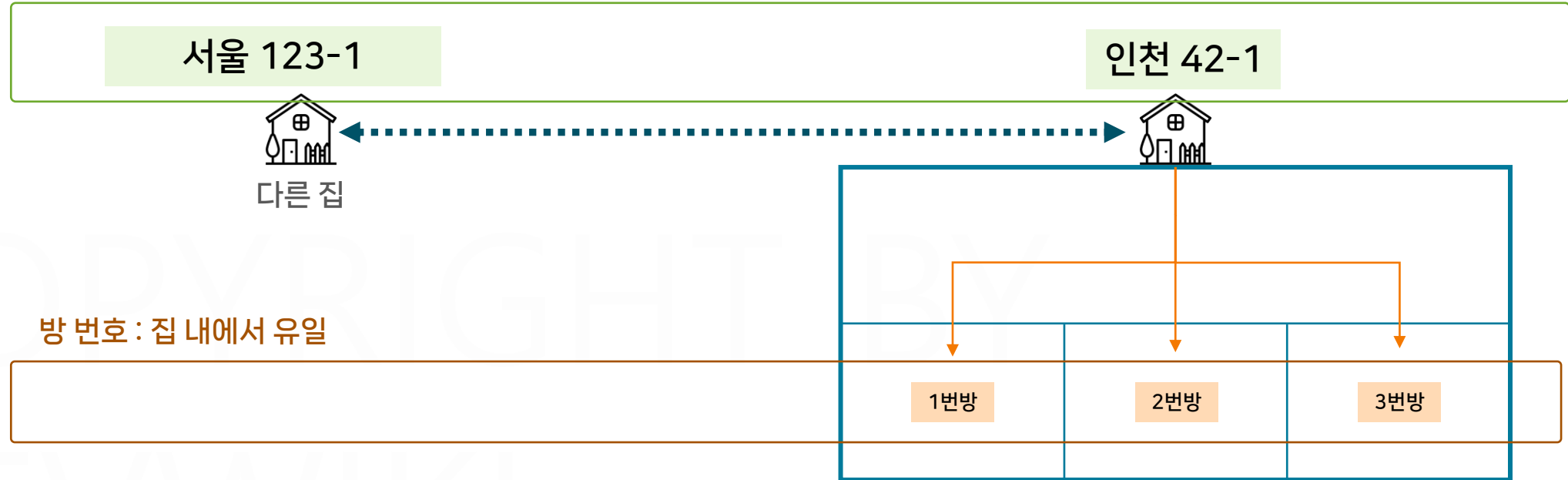
PART6. 네트워크

PART6. 네트워크

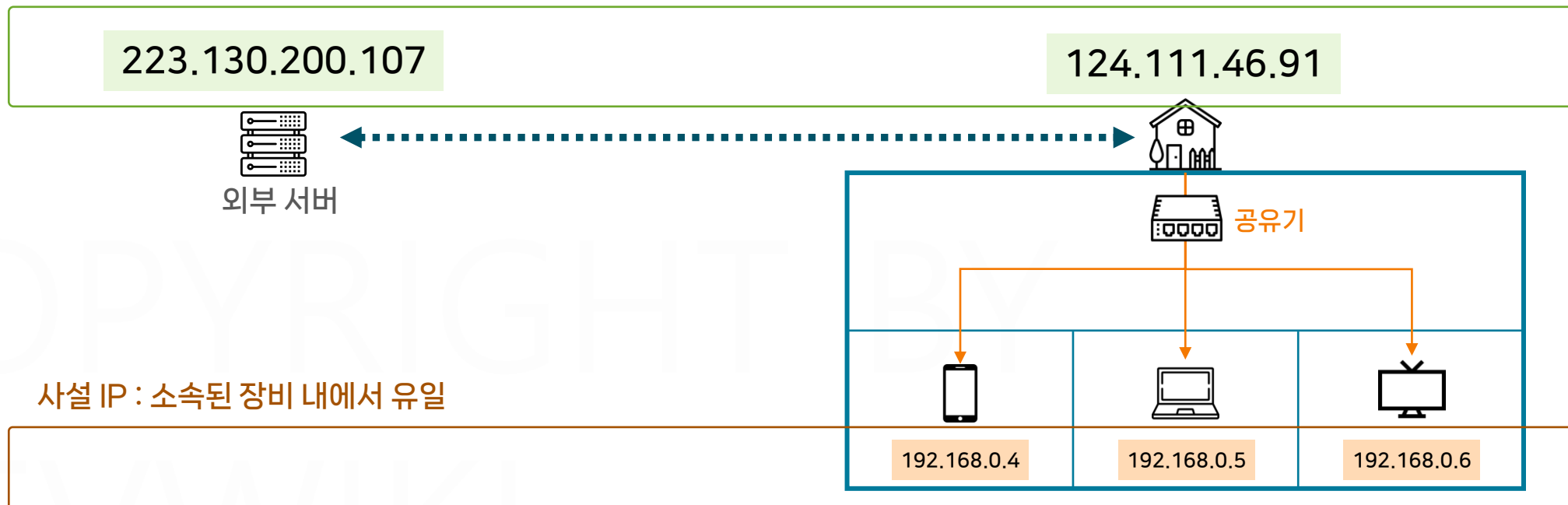
네트워크 기본



집 주소 : 전 세계에서 유일



공인 IP : 전 세계에서 유일



사설 IP 주소 대역

10.0.0.0 – 10.255.255.255 (Class A)

172.16.0.0 – 172.31.255.255 (Class B)

192.168.0.0 – 192.168.255.255 (Class C)

공인망 : 외부 네트워크를 사용한 통신망

223.130.200.107



124.111.46.91

라우터(공인IP->사설IP)

사설망 : 내부 네트워크 통신망

서버

192.168.0.4

192.168.0.5

192.168.0.6

192.168.0.7

124.111.46.91

라우터(공인IP->사설IP)

서버

192.168.0.4
192.168.0.5

192.168.0.6

192.168.0.7

192.168.0.8

서버

192.168.0.4

랜선

192.168.0.5

192.168.0.5:80
192.168.0.5:53
192.168.0.5:8080
192.168.0.5:3000

포트

네트워크
인터페이스

공인IP와 사설IP를 랜덤하게 매핑

NAT테이블 예시

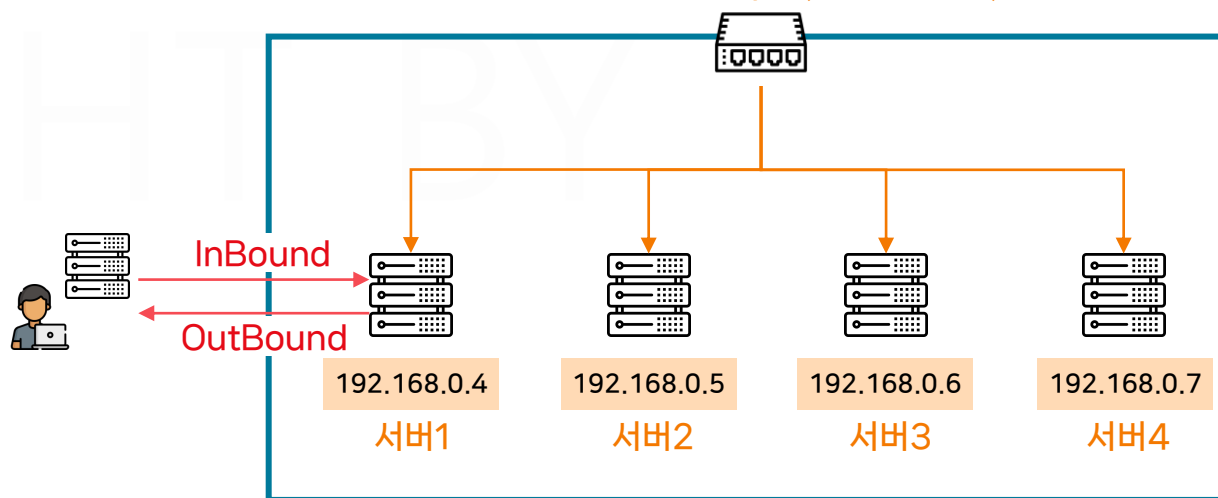
공인IP주소	공인포트번호	사설IP주소	사설포트번호
124.111.46.91	10001	192.168.0.4	80
124.111.46.91	10002	192.168.0.5	5432

223.130.200.107



124.111.46.91

라우터(공인IP->사설IP)



공인IP와 사설IP를 사용자가 지정하여 매핑

포트포워딩 예시

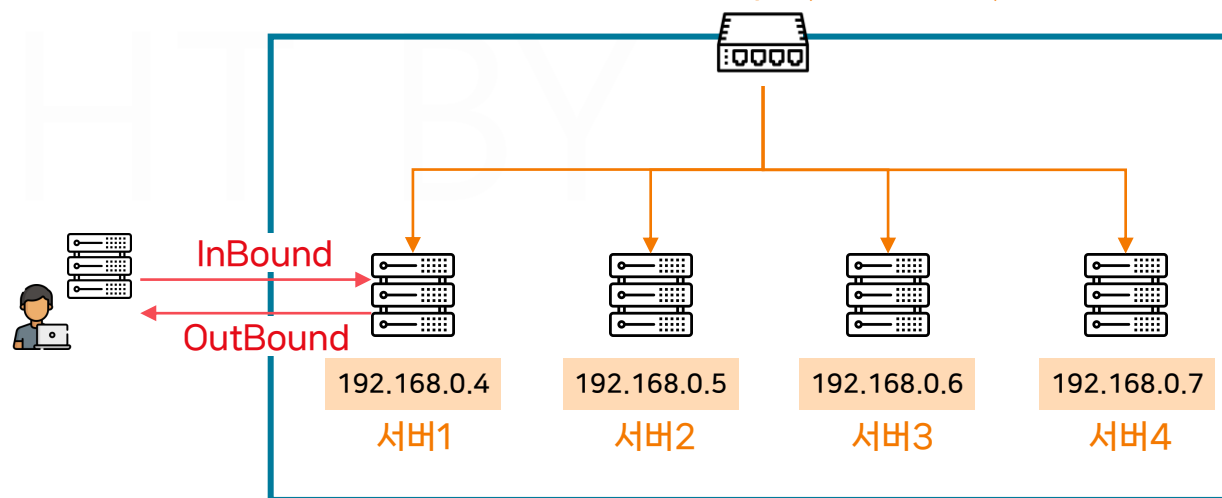
공인IP주소	공인포트번호	사설IP주소	사설포트번호
124.111.46.91	80	192.168.0.4	80
124.111.46.91	21	192.168.0.5	20021

223.130.200.107



124.111.46.91

라우터(공인IP->사설IP)



Domain	IPAddr
devwiki-docker.com	124.111.46.91
...	...



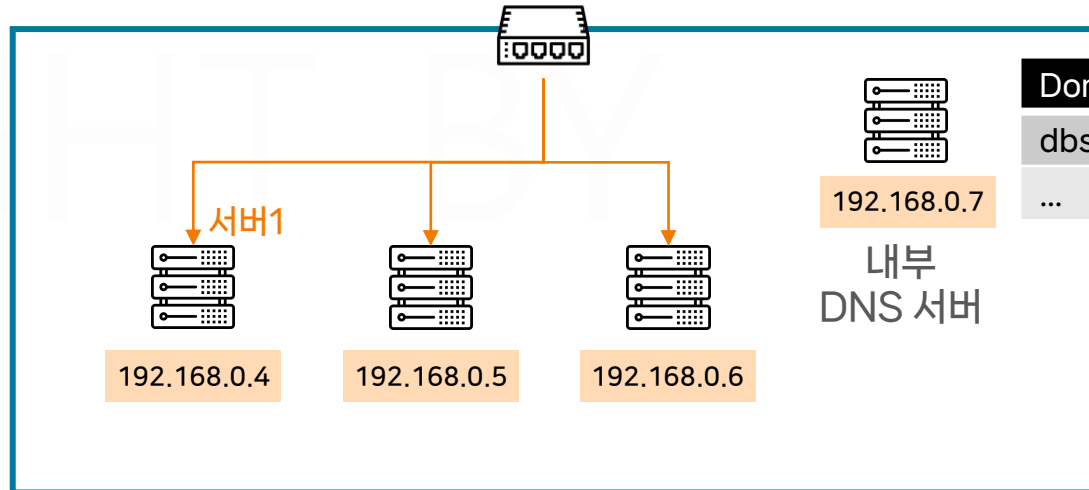
DNS 서버
8.8.8.8

223.130.200.107

124.111.46.91


외부 서버

라우터(공인IP->사설IP)



Domain	IPAddr
dbserver	192.168.0.6
...	...

PART6. 네트워크

도커 네트워크

공인망 : 외부 네트워크를 사용한 통신망

223.130.200.107



외부 서버

124.111.46.91

라우터(공인IP->사설IP)



사설망 : 내부 네트워크 통신망

물리 네트워크

서버



192.168.0.4



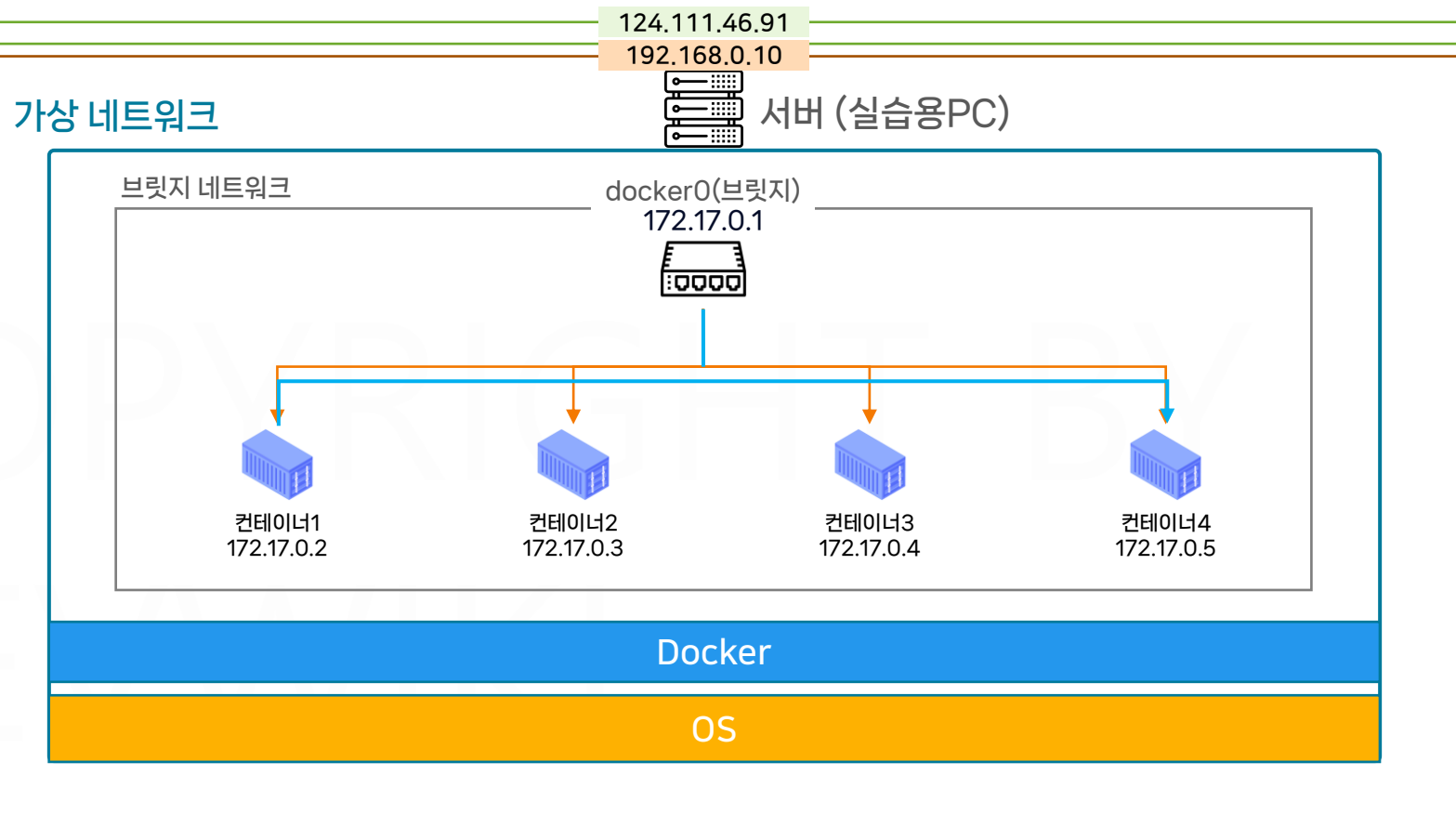
192.168.0.5



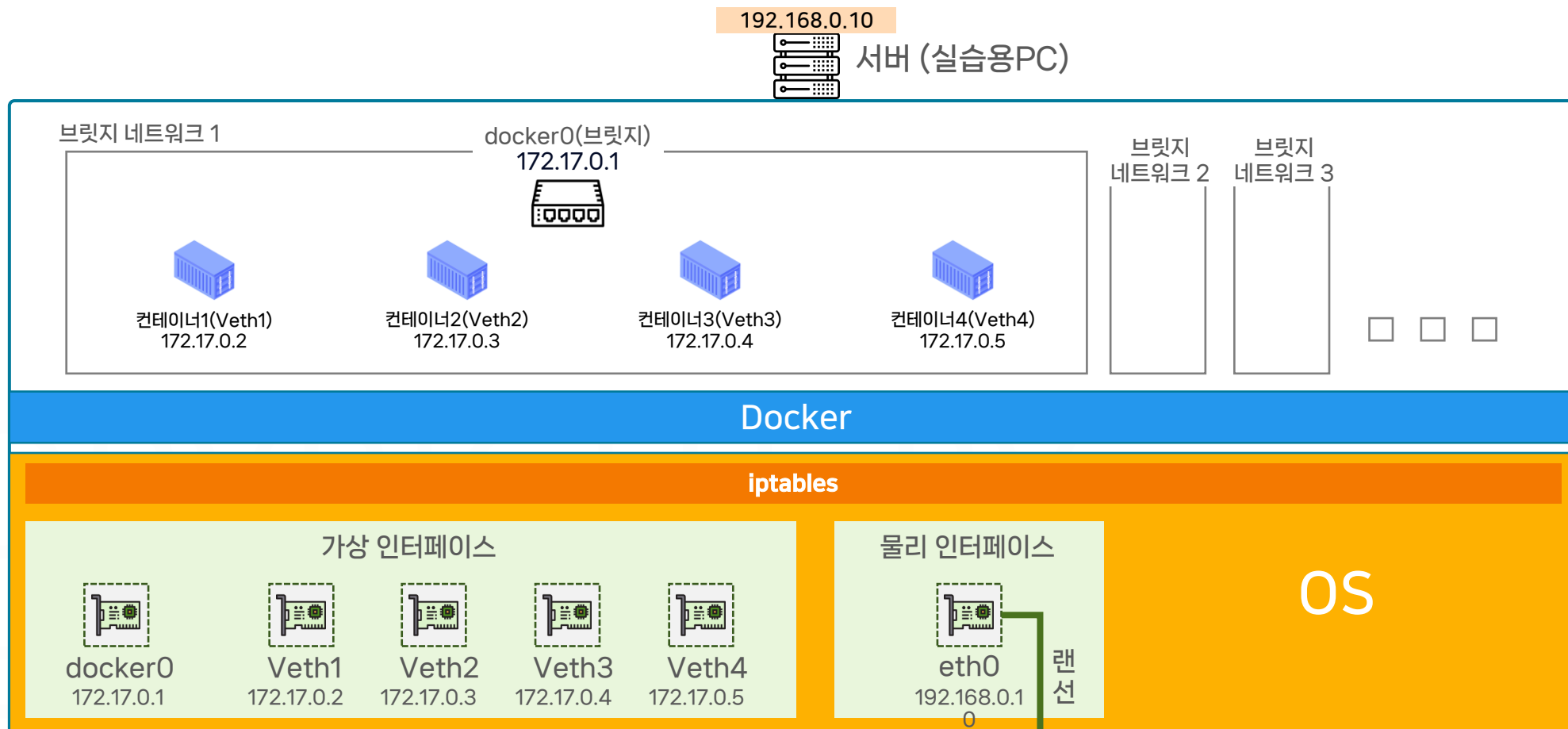
192.168.0.6



192.168.0.7

공인망
사설망

1. 가상의 네트워크 브릿지 생성
2. 컨테이너에 가상의 IP 할당
3. 컨테이너간의 통신 전달



- iptables: Linux 내부의 네트워크 트래픽 관리 및 제어
- 컨테이너 1개 당 Host머신에 가상의 Veth 인터페이스 1개 생성
- Docker가 iptables의 규칙을 수정하여 Veth 인터페이스의 트래픽 제어

docker network ls

네트워크 리스트 조회

docker network inspect 네트워크명

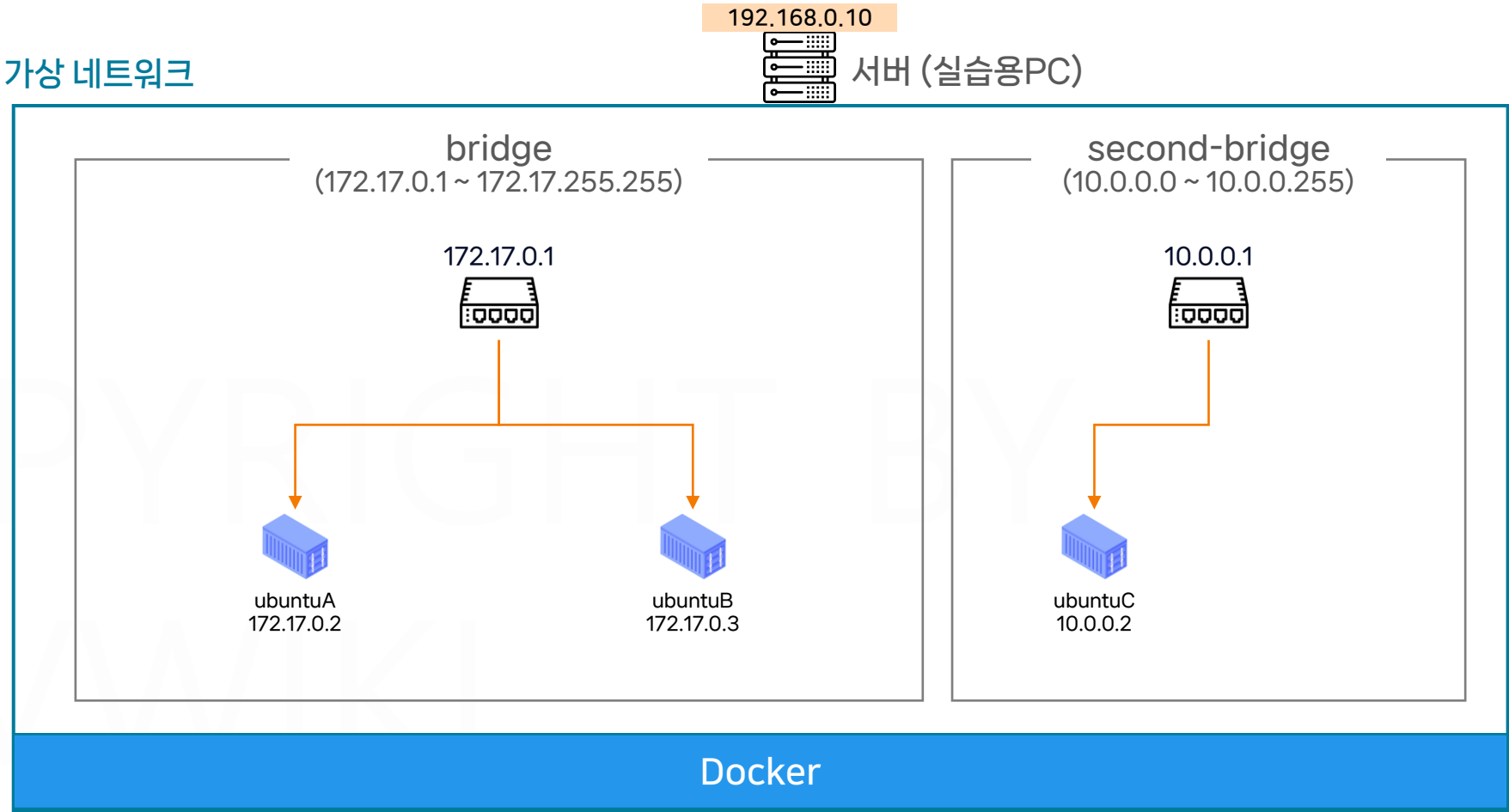
네트워크 상세 정보 조회

docker network create 네트워크명

네트워크 생성

docker network rm 네트워크명

네트워크 삭제



네트워크 및 IP 확인 (Shell1)

1. 네트워크 리스트 조회

```
docker network ls
```

```
docker network inspect bridge
```

2. 새로운 네트워크 생성

```
docker network create --driver bridge --subnet 10.0.0.0/24 --gateway 10.0.0.1 second-bridge
```

```
docker network ls
```

```
docker network inspect second-bridge
```

[Shell2로 이동](#) ==>>(다음 페이지)

5. 컨테이너의 네트워크 확인

```
docker ps
```

```
docker container inspect ubuntuA
```

```
docker container inspect ubuntuB
```

```
docker container inspect ubuntuC
```

[Shell2로 이동](#) ==>>(다음 페이지)

ubuntuA 컨테이너 생성 (Shell2)

3. ubuntuA, ubuntuB 컨테이너 생성 후 ubuntuB연결

```
docker run -d --name ubuntuA devwikirepo/pingbuntu
```

```
docker run -it --name ubuntuB devwikirepo/pingbuntu bin/bash
```

Shell3로 이동 ==>

6. ubuntuA의 IP로 Ping 테스트

```
# ping 172.17.0.2
```

7. ubuntuC의 IP로 Ping 테스트

```
# ping 10.0.0.2
```

Shell3로 이동 ==>

ubuntuA 컨테이너 생성 (Shell3)

4. ubuntuC 컨테이너를 second-bridge 네트워크에 생성

```
docker run -it --network second-bridge --name ubuntuC devwikirepo/pingbuntu bin/bash
```

<=== Shell1로 이동(전페이지)

8. ubuntuA의 IP로 Ping 테스트

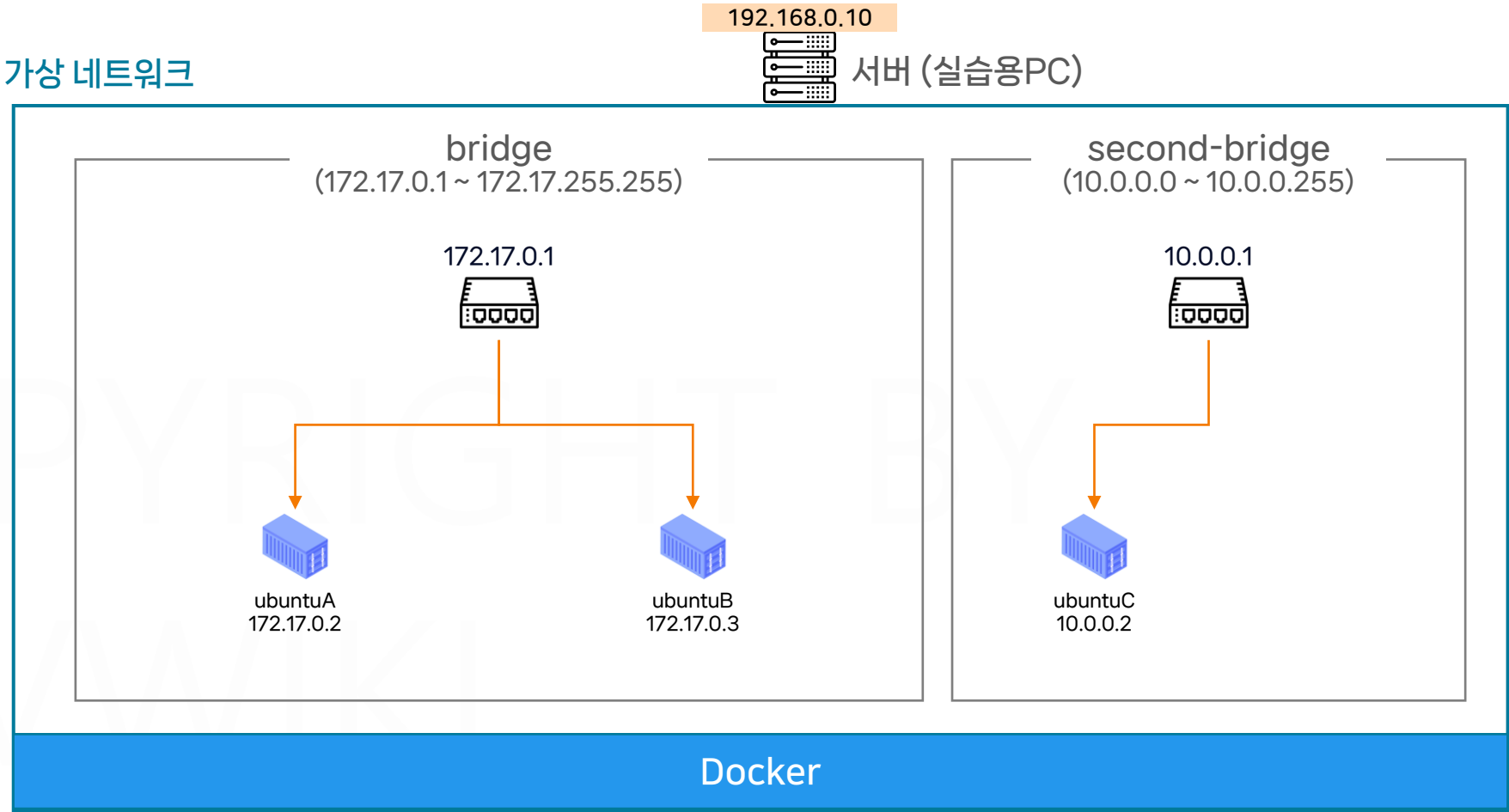
```
# ping 172.17.0.2
```

9. Ctrl+C로 탈출 후 컨테이너 및 네트워크 삭제

```
docker rm -f ubuntuA ubuntuB ubuntuC
```

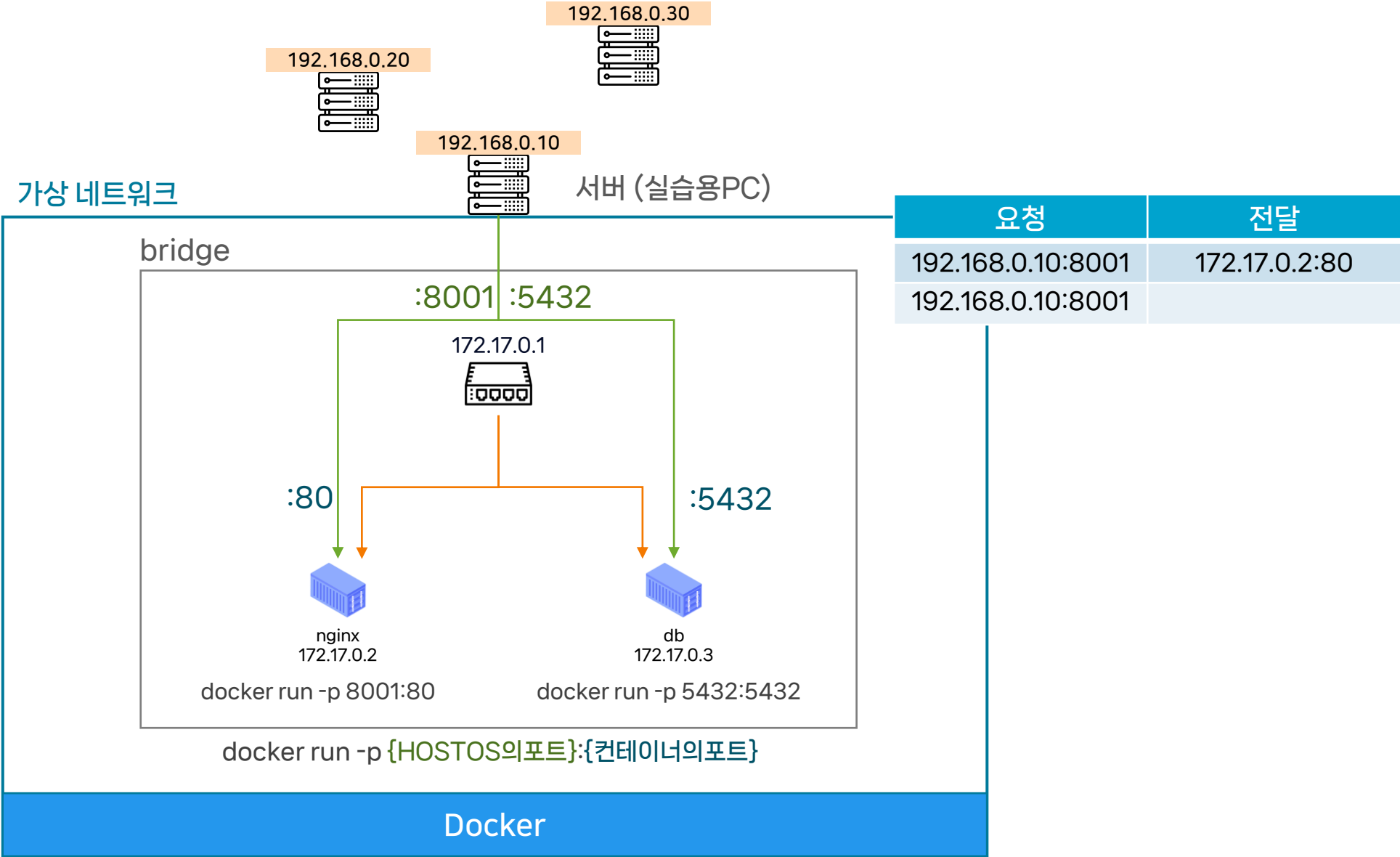
```
docker network rm second-bridge
```

==> 실습 종료



`docker run -p` HostOS의 포트: 컨테이너의 포트

HostOS로의 네트워크 접근을 컨테이너로 포트포워딩



포트포워딩 실습

1. 포트포워딩 없이 nginx 실행

```
docker run -d --name nginx nginx
```

```
docker container inspect nginx
```

2. 8001 포트에 nginx 실행 및 localhost:8001 접속

```
docker run -d -p 8001:80 --name nginx2 nginx
```

3. 8002 포트에 redapp 실행 및 127.0.0.1:8002 접속

```
docker run -d -p 8002:3000 --name redColorApp --env COLOR=red devwikirepo/envnodecolorapp
```

4. 8003 포트에 blueapp 실행 및 IP주소:8003 접속

```
docker run -d -p 8003:3000 --name blueColorApp --env COLOR=blue devwikirepo/envnodecolorapp
```

5. 8003 포트에 yelloapp 실행 시도

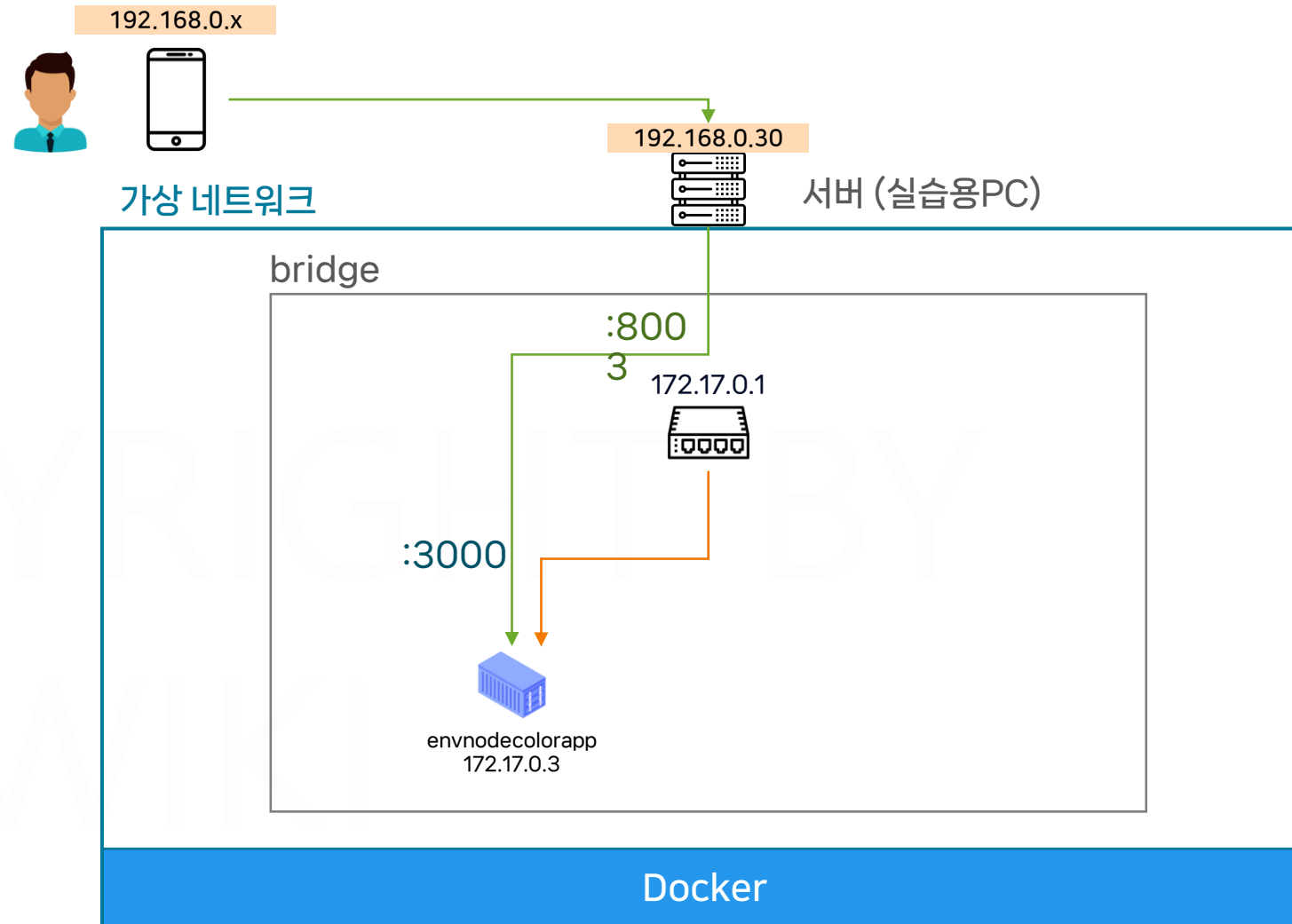
```
docker run -d -p 8003:3000 --name yellowColorApp --env COLOR=yellow devwikirepo/envnodecolorapp
```

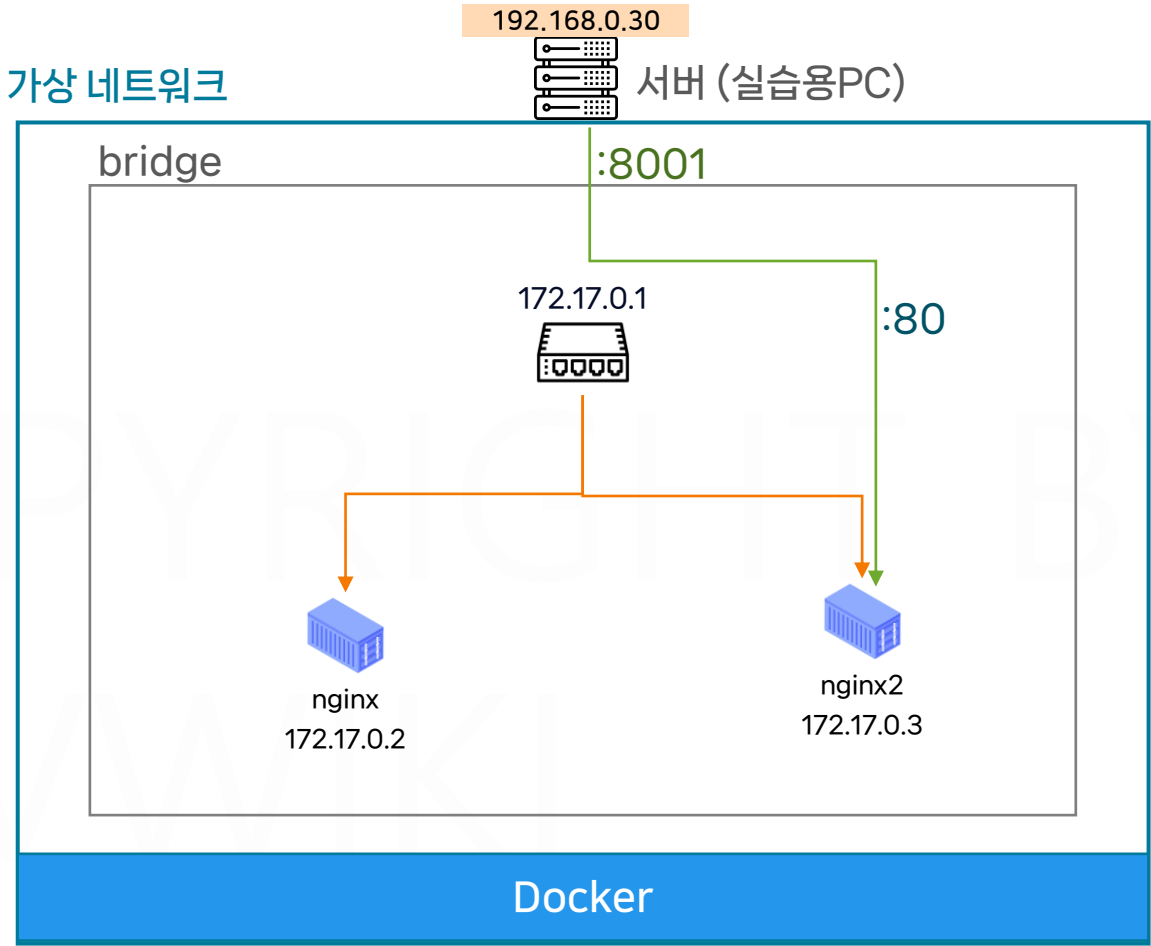
6. 8004 포트에 greenapp 실행 및 접속(애플리케이션과 다른 포트)

```
docker run -d -p 8004:3030 --name greenColorApp --env COLOR=green devwikirepo/envnodecolorapp
```

6. 컨테이너 삭제

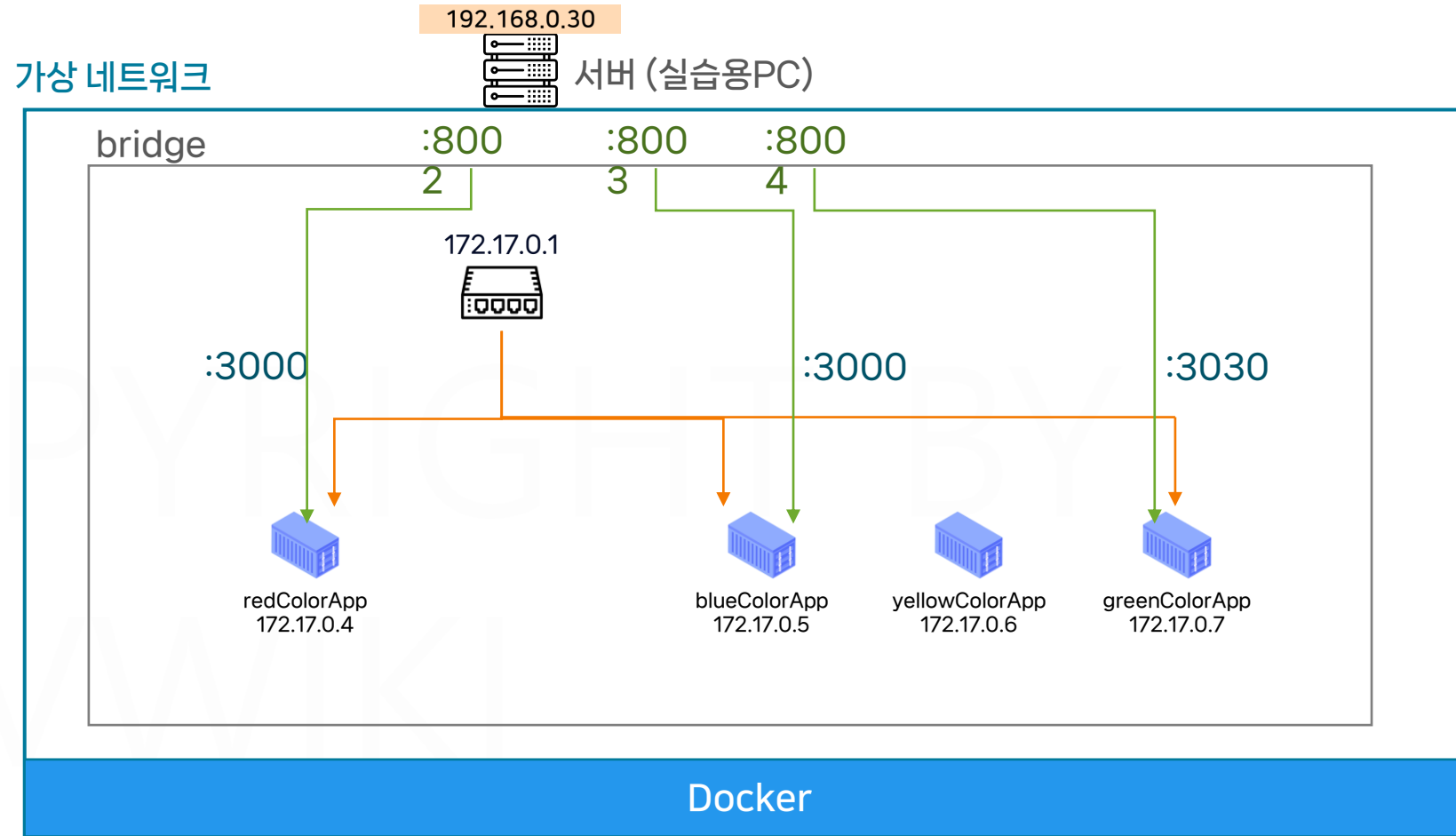
```
docker rm -f nginx redColorApp blueColorApp yellowColorApp greenColorApp
```

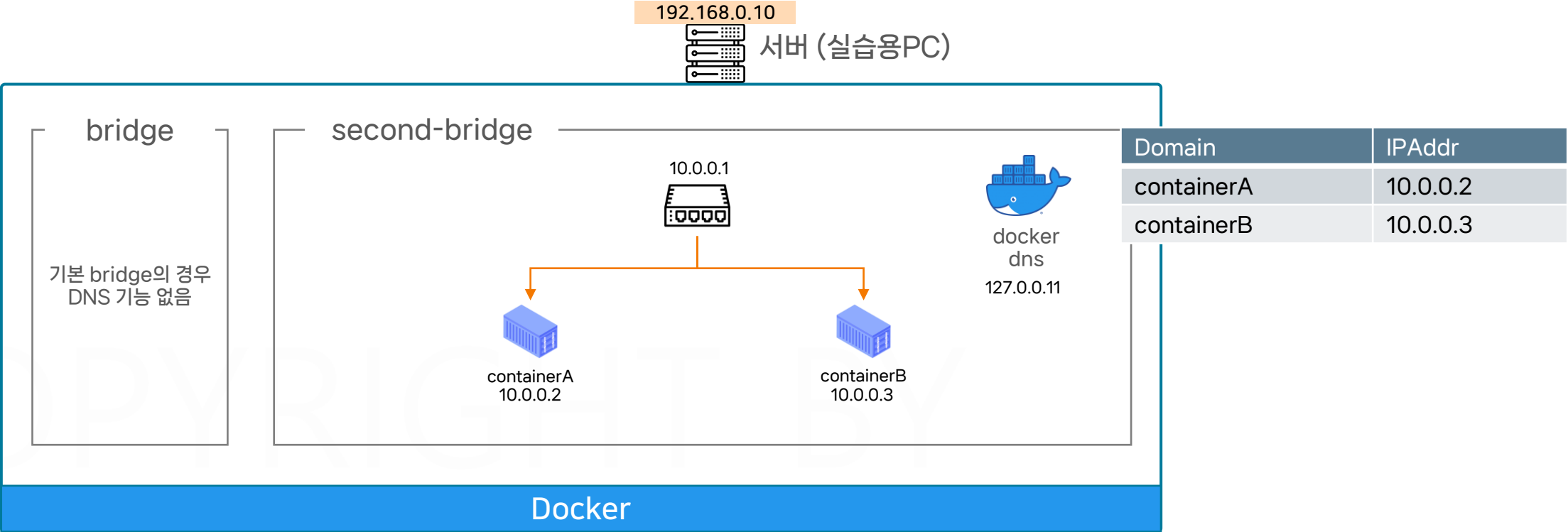




내부 접근 시
localhost:8001
127.0.0.1:8001
192.168.0.30:8001

외부 접근 시
192.168.0.30:8001





DNS 실습 (Shell1)

1. 새로운 네트워크 생성

```
docker network create --driver bridge --subnet 10.0.0.0/24 second-bridge
```

2. containerA 실행 및 DNS서버 주소 확인

```
docker run -it --network second-bridge --name containerA devwikirepo/pingbuntu bin/bash  
cat /etc/resolv.conf
```

3. 핑테스트

```
ping containerB
```

Shell2로 이동 ==>

5. containerB 실행 후 Ping 테스트

```
ping containerB
```

Shell2로 이동 ==>

7. containerB 삭제 후 Ping 테스트

```
ping containerB
```

Shell2로 이동 ==>

DNS 실습 (Shell2)

4. containerB 실행

```
docker run -d --network second-bridge --name containerB devwikirepo/pingbuntu  
docker inspect containerB
```

<==== Shell1로 이동

6. containerB 삭제

```
docker rm -f containerB
```

<==== Shell1로 이동

8. containerA 및 network 삭제

```
docker rm -f containerA  
docker network rm second-bridge
```

====> 실습 종료

브릿지 네트워크(Bridge) : **도커 브릿지**를 활용해 컨테이너간 통신, NAT 및 포트포워딩 기술을 활용해 외부 통신 지원

호스트 네트워크(Host) : 호스트의 네트워크를 공유, 모든 컨테이너는 **호스트 머신과 동일한 IP**를 사용, 포트 중복 불가능

오버레이 네트워크(Overlay) : Kubernetes에서 사용, **호스트 머신이 다수일 때 네트워크 관리** 기술

Macvlan 네트워크 : 컨테이너에 MAC 주소를 할당하여 **물리 네트워크 인터페이스에 직접 연결**

PART6. 네트워크

Leafy 네트워크

Leafy 애플리케이션 구성 및 실행

```
docker network create leafy-network
```

=> 애플리케이션의 네트워크 생성

```
docker run -d --name leafy-postgres --network leafy-network devwikirepo/leafy-postgres:1.0.0
```

=> 네트워크 지정

```
docker run -d -p 8080:8080 -e DB_URL=leafy-postgres --network leafy-network --name leafy devwikirepo/leafy-backend:1.0.0
```

=> 8080포트 포트포워딩 => DB접속 URL을 postgres 컨테이너의 이름으로 지정

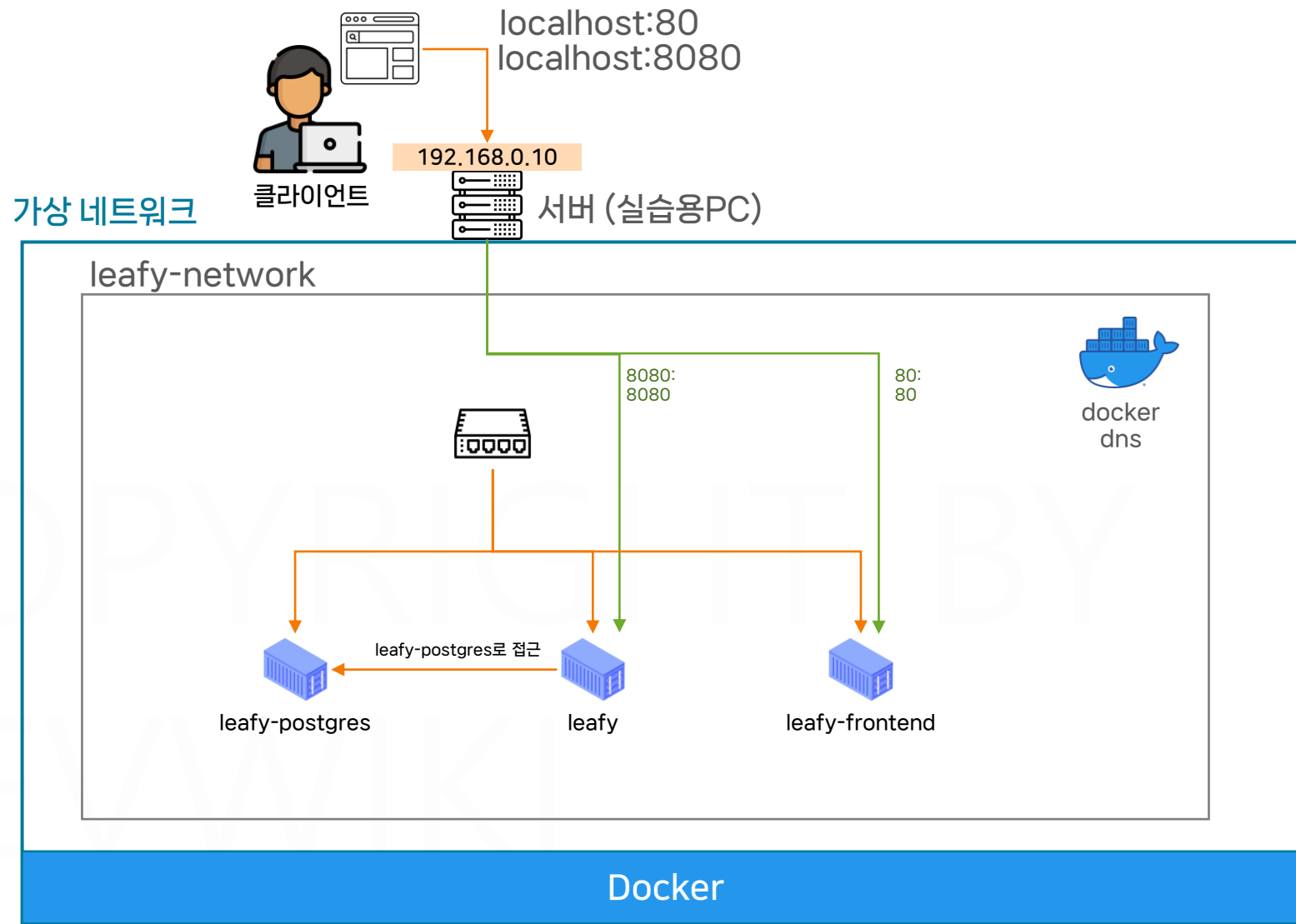
```
docker run -d -p 80:80 --network leafy-network --name leafy-front devwikirepo/leafy-frontend:1.0.0
```

=> 80포트 포트포워딩

```
docker ps
```

2. 구성 환경 삭제

```
docker rm -f leafy-front leafy leafy-postgres
```



1. 프론트엔드 컨테이너는
HostOS의 80 포트로 접속

2. 백엔드 컨테이너는
HostOS의 8080포트로 접속

3. 데이터베이스 컨테이너는
포트포워딩이 없어 외부에서 접근
불가,
내부 DNS서버에 등록된 레코드를
통해 백엔드 컨테이너에서 leafy-
postgres로 접속